SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _____M Vineela (192371053) _____ (Name Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Vineela*

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate

with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search
ii. Analyze the challenges of partial observability and dynamic environments
iii. Describe how the rover builds and updates its internal model
iv. Compare online search with uninformed and informed search strategies
v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation
ii. Explain how backtracking search works in this scenario
iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem
ii. Analyze the computational challenges of large game trees
iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
iv. Design an evaluation function and justify the factors considered
v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

i. Explain how Greedy Best-First Search behaves in this scenario and discuss its strengths and weaknesses under uncertainty.

Greedy Best-First Search (GBFS) is an informed search algorithm that selects the node that appears closest to the goal using a heuristic function.

The evaluation function is:

[f(n)=h(n)]

Where (h(n)) represents the estimated straight-line distance from the current location to the destination.

Working:

1. The drone identifies its current location and the destination.
2. It calculates the estimated distance from nearby locations to the destination.
3. It selects the location with the lowest heuristic value.
4. It continues searching toward the destination.
5. If a path becomes blocked, it searches for an alternative route using the available information.

Example: Suppose the drone can travel through Route A or Route B. Route A appears closer to the destination, but Route B is safer and has less weather-related delay. GBFS may select Route A because it considers the heuristic distance rather than the complete travel cost.

Strengths:

- Finds promising routes quickly.
- Often requires less computation than exhaustive search.
- Useful when emergency deliveries require quick decisions.

Weaknesses:

- Ignores the actual cost already incurred.
- May select a short but dangerous route.
- Can perform poorly when weather and terrain change unexpectedly.
- Does not guarantee an optimal solution.

ii. Explain how A* handles the same situation, including how it balances cost and heuristic.

A* is an informed search algorithm that considers both the actual cost of reaching a location and the estimated cost of reaching the destination.

Its evaluation function is:

[f(n)=g(n)+h(n)]

Where:

- (g(n)) = actual cost from the starting point to the current node.
- (h(n)) = estimated remaining cost to the destination.
- (f(n)) = estimated total cost of the route through that node.

Working:

1. The drone identifies its starting point and destination.
2. It calculates actual travel costs based on time, terrain, weather, and risk.
3. It estimates the remaining cost using the heuristic.
4. It selects the node with the lowest estimated total cost.
5. It updates the costs when new information becomes available.
6. If a path becomes blocked, it recalculates the route and selects an alternative.

Example:

Factor	**Route A**	**Route B**
Actual cost (g(n))	12	7
Estimated cost (h(n))	3	6
Total cost (f(n))	15	13

A* selects Route B because its estimated total cost is lower.

Advantages:

- Balances actual and estimated costs.
- Can find an optimal route under suitable conditions.
- Can consider weather, terrain, and risk in its cost function.

Limitation: A* may require more memory and computation than GBFS. Its optimality guarantee depends on factors such as non-negative movement costs, an admissible heuristic, and correct handling of the search graph.

iii. Analyze the impact of heuristic accuracy on both algorithms.

A heuristic estimates the remaining cost from a current location to the destination.

- Accurate heuristic: Helps the algorithm identify promising routes.
- Overestimated heuristic: May lead to poor decisions. In A*, it can remove the guarantee of optimality.
- Underestimated heuristic: A* can still guarantee an optimal solution if the heuristic is admissible and the algorithm is implemented correctly.
- Unreliable heuristic: Can cause inefficient or unsafe route choices if it fails to reflect obstacles and environmental conditions.

For example, straight-line distance may be misleading when a flooded area or strong wind makes the direct route costly.

GBFS depends entirely on the heuristic for ranking nodes. A* also considers the actual cost already incurred, making it less dependent on the heuristic alone.

iv. Discuss how dynamic changes, such as blocked paths, affect search performance.

Dynamic changes make route planning more difficult because the environment may change while the drone is travelling.

- Previously available routes may become blocked.
- Weather may increase travel time and risk.
- New obstacles may make a planned route unsafe.
- Repeated replanning may increase computation and delay delivery.

GBFS may quickly choose an alternative route but can repeatedly select locally attractive routes that are poor overall.

A* can update route costs and search for a better alternative. However, repeatedly restarting the search may be expensive. Algorithms such as D Lite* can reuse previous search information to handle changing paths more efficiently.

v. Recommend the most suitable algorithm and justify your decision.

Recommendation: A with real-time replanning.*

A* is more suitable because the drone must consider travel time, terrain, weather, and safety instead of simply moving toward the destination.

Reasons:

1. Optimality: It can find a minimum-cost route when the required conditions are satisfied.

2. Safety: Risk can be included in the cost function, while unsafe paths can be excluded.
3. Adaptability: The route can be updated when new information arrives.
4. Efficiency: The heuristic helps reduce unnecessary exploration.

For frequent environmental changes, D* Lite or another incremental replanning algorithm may be more efficient. Real-time obstacle detection and safety constraints must also be used.

Conclusion: A* with real-time replanning is the best general choice for balancing route quality, urgency, and safety.

2. Smart City Traffic Signal Optimization

Formulate the problem as an optimization problem and explain why traditional search is inefficient.

The objective is to optimize traffic signals across hundreds of intersections while minimizing total waiting time, fuel consumption, and traffic congestion.

The objective function is:

[min J=w_1W+w_2F+w_3C]

Where:

- (W) = total vehicle waiting time.
- (F) = total fuel consumption.
- (C) = traffic congestion measure.
- (w_1,w_2,w_3) = weights representing the importance of each objective.

The decision variables include green-light duration, red-light duration, signal phase order, and coordination between intersections.

Constraints:

- Minimum and maximum signal durations.
- Pedestrian crossing requirements.
- Traffic safety rules.
- Coordination between neighboring intersections.
- Emergency vehicle priority.

Traditional exhaustive search is inefficient because the number of possible signal combinations increases rapidly with the number of intersections and available

timings. Traffic conditions also change continuously, making previously calculated solutions outdated.

i. Explain how Hill Climbing works in this scenario and identify its limitations.

Hill Climbing is a local search algorithm that repeatedly moves from the current solution to a better neighboring solution.

Working:

1. Generate an initial signal timing plan.
2. Calculate its waiting time, fuel consumption, and congestion.
3. Modify one or more signal timings slightly.
4. Evaluate the new plan.
5. Accept the change if it improves the objective.
6. Repeat until no better neighboring solution is found.

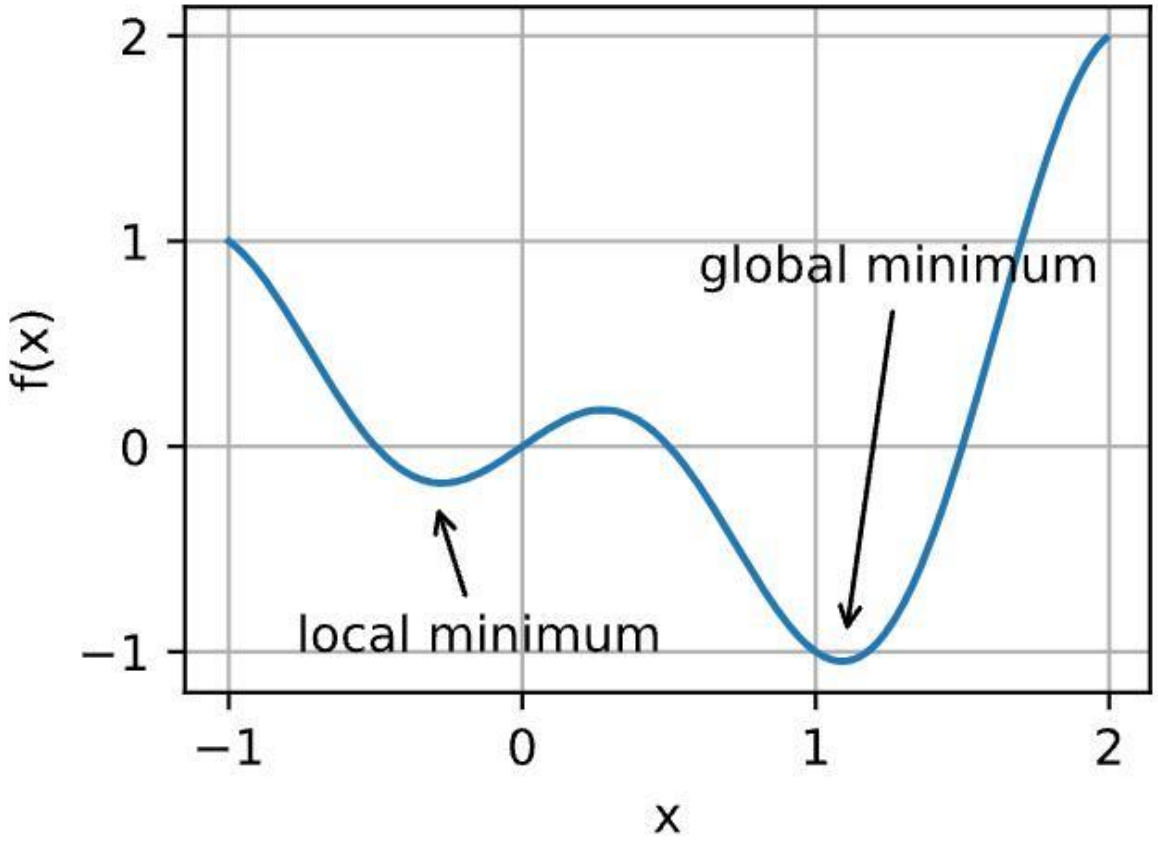
Example: If increasing a junction's green-light duration from 30 seconds to 40 seconds reduces vehicle waiting time, Hill Climbing accepts the change.

Limitations:

- Can become trapped in a local optimum.
- May stop on a plateau where neighboring solutions have similar scores.
- May fail when several intersections require coordinated changes.
- Does not guarantee a global optimum.

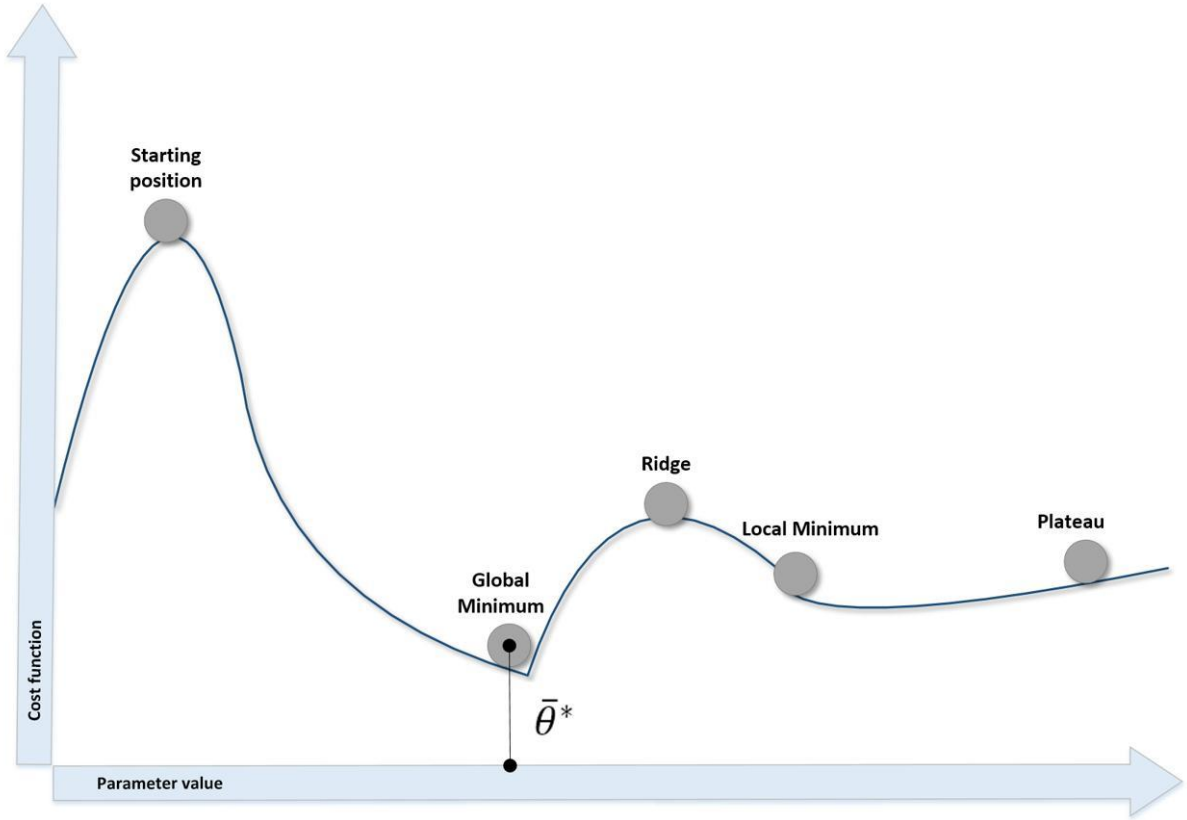
ii. Discuss local maxima, plateaus, and ridges with real-world interpretation.

For a minimization problem, these concepts are commonly discussed as local minima, although they can also be expressed using a reversed fitness score.



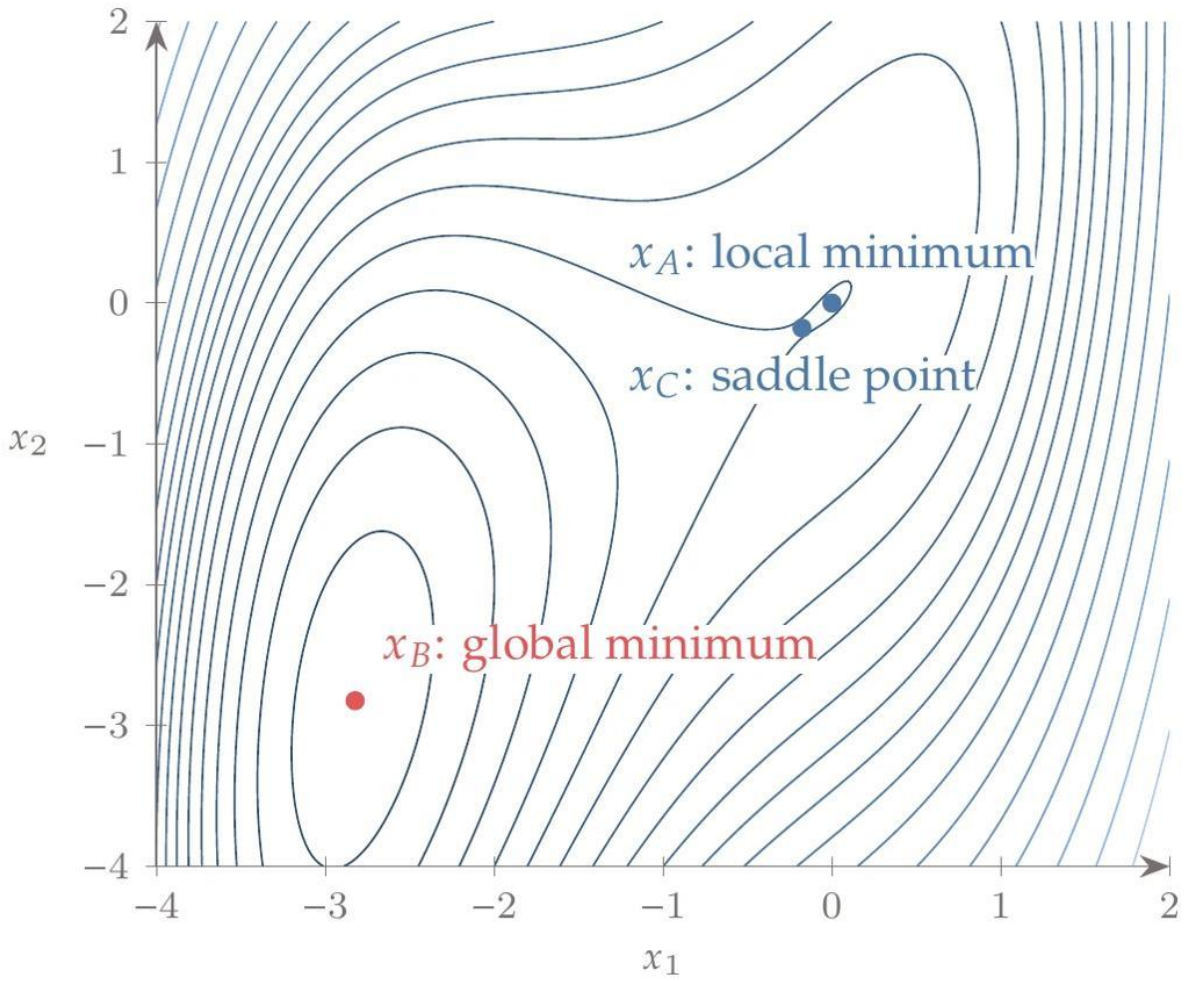
1. Local optimum

A signal timing plan reduces congestion in one area but increases it elsewhere. Small changes do not appear to improve the total score, even though a better overall solution exists.



2. Plateau

Several signal timing plans produce nearly identical results. Hill Climbing cannot easily determine which direction will lead to improvement.



3. Ridge

A good solution requires coordinated changes to multiple intersections. Changing only one signal at a time may not improve traffic, making progress difficult.

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations.

A. Simulated Annealing (SA)

Simulated Annealing is inspired by the cooling process of metals. It sometimes accepts a worse solution to escape a local optimum.

Working:

1. Start with an initial signal timing plan.
2. Generate a neighboring plan.
3. Accept the new plan if it improves the objective.
4. Sometimes accept a worse plan according to a probability based on temperature.

5. Gradually reduce the temperature.
6. Return the best solution found.

For a minimization problem, the probability of accepting a worse solution is:

[P=e^{-Delta ET}]

Where (Delta E) is the increase in cost and (T) is the temperature.

SA can escape local optima and is relatively simple to implement. However, its performance depends on the cooling schedule, and excessive exploration may be too slow for urgent traffic changes.

B. Genetic Algorithms (GA)

Genetic Algorithms are inspired by natural selection and evolution.

Working:

1. Create a population of possible signal timing plans.
2. Evaluate each plan using the objective function.
3. Select better-performing plans as parents.
4. Apply crossover to combine parts of different plans.
5. Apply mutation to introduce new timing variations.
6. Repeat until a stopping condition or time limit is reached.

For example, one plan may coordinate signals on a main road, while another reduces delays at cross streets. Crossover can combine useful features from both.

GA explores multiple solutions simultaneously and is suitable for large, complex search spaces. However, it can require substantial computation and careful parameter tuning.

iv. Recommend the best approach based on scalability and adaptability.

Recommendation: Genetic Algorithm combined with real-time traffic feedback.

Reasons:

- Explores multiple solutions instead of following one local path.
- Can optimize many interconnected signal timings.
- Crossover and mutation help explore different combinations.
- Can adapt by updating its population using current traffic data.

For faster local adjustments, Hill Climbing can refine the best solution produced by GA.

Conclusion: A hybrid GA and local optimization approach is suitable for large-scale traffic management. Computation must be bounded, and safety constraints must be enforced. Simulated Annealing is a reasonable alternative when a simpler optimization method is preferred.

3. Autonomous Rover Exploring Mars

i. Explain why this problem requires an online search agent instead of offline search.

An offline search agent plans using a model of the environment before taking action. It generally assumes that enough information is available to calculate a route in advance.

An online search agent interleaves planning, action, observation, and learning.

The Mars rover has an incomplete map and limited sensing capability. It may discover a crater, rock, or unexpected terrain only after reaching a new location.

Therefore, it cannot depend entirely on a route calculated in advance.

Working of an online search agent:

1. Observe the surrounding terrain.
2. Update the internal map.
3. Select a safe action.
4. Move to the selected location.
5. Observe newly discovered conditions.
6. Update the map and select the next action.

This process allows the rover to adapt its plan as it explores.

ii. Analyze the challenges of partial observability and dynamic environments.

Partial observability

The rover cannot observe the entire environment at once. Its sensors provide only limited information about nearby terrain.

Challenges include:

- Hidden craters and obstacles.
- Uncertain distances and terrain conditions.
- Incomplete knowledge of safe paths.
- Sensor errors and limited visibility.

Dynamic environments

Although Mars generally changes more slowly than a busy road network, dust storms, shifting sand, changing illumination, and equipment degradation can affect navigation.

The rover must therefore make decisions using incomplete and sometimes outdated information.

iii. Describe how the rover builds and updates its internal model.

The rover maintains an internal representation of its surroundings, such as a grid map.

Each grid cell may represent:

- Known safe terrain.
- A detected obstacle.
- Unknown terrain.
- Terrain with uncertain movement cost.

Steps:

1. Collect sensor information using cameras and other instruments.
2. Estimate the current position.
3. Update the map using new observations.
4. Assign movement costs based on terrain and hazard estimates.
5. Plan a route through known safe regions.
6. Move and gather further information.
7. Replan if new observations invalidate the route.

Example: If the rover initially believes an area is safe but later detects a crater, it marks the region as hazardous and searches for an alternative route.

iv. Compare online search with uninformed and informed search strategies.

Feature	**Uninformed search**	**Informed search**	**Online search**
Knowledge	Does not require a heuristic	Uses a heuristic	Uses observations and learned information
Examples	BFS, DFS	A*, Greedy Best-First	Online exploration and replanning

Map requirement	Usually requires a defined search graph	Usually requires a defined search graph and cost model	Can operate with an incomplete map
Adaptability	Limited without restarting or modifying search	Can replan when information changes	Naturally interleaves action and learning
Rover suitability	Useful for simple, known maps	Useful for route planning with reliable information	Suitable for exploring unknown terrain

Online search is not entirely separate from informed and uninformed search. An online agent can use an informed technique such as A* whenever its available model supports it.

v. Justify the most suitable approach considering uncertainty, adaptability, and safety.

Recommendation: Online search with heuristic-guided planning and continuous replanning.

A practical rover can use:

- Online exploration to discover unknown regions.
- A-based planning* on the currently known map.
- Local obstacle avoidance to respond to immediate hazards.
- Incremental replanning when the map changes.

This combination provides adaptability while supporting efficient route planning.

Safety must take priority over the shortest route. The rover should avoid hazardous regions and use a safe fallback behavior when sensor information is unreliable.

Conclusion: An online search agent is the most suitable choice because the rover must continuously learn about its surroundings and make decisions without a complete map.

4. University Exam Timetabling Using Constraint Satisfaction Problems (CSP)

Formulate the problem as a CSP.

A Constraint Satisfaction Problem (CSP) consists of three components:

[CSP=(X,D,C)]

Where:

- (X) = variables.

- (D) = domains of possible values.
- (C) = constraints that restrict valid combinations.

The goal is to assign a valid time and examination hall to every exam while satisfying all hard constraints.

i. Clearly define variables, domains, and constraints.

1. Variables

Each examination is represented by a variable.

For example:

- (X_1) = Mathematics exam.
- (X_2) = Computer Networks exam.
- (X_3) = Artificial Intelligence exam.

Each variable can represent a combination of an examination slot and a hall.

2. Domains

A domain contains all possible assignments for each variable.

For example:

[D(X_1)={(9AM,H1),(9AM,H2),(2PM,H1)}]

Here, (H1) and (H2) represent examination halls. In practice, the domain contains only slots and halls that satisfy basic availability requirements.

3. Constraints

Constraint	**Explanation**
No overlapping exams	Two exams attended by the same student cannot occur at the same time.
Hall capacity	The hall must accommodate the students taking the exam.
Hall availability	A hall cannot host two exams simultaneously.
Subject order	A prerequisite exam must occur before a dependent exam if required.
Faculty availability	Exams requiring the same invigilator must respect their availability.

Special accommodations	Suitable rooms, access arrangements, and time allowances must be provided.

The objective is to find an assignment satisfying all hard constraints. Preferences, such as avoiding too many exams on consecutive days, can be handled as soft constraints.

ii. Explain how backtracking search works.

Backtracking is a systematic search technique that assigns values to variables and reverses assignments when a conflict occurs.

Steps:

1. Select an exam that has not yet been scheduled.
2. Choose a possible time slot and hall.
3. Check whether the assignment satisfies all applicable constraints.
4. If valid, assign the slot and continue with the next exam.
5. If no valid slot is available, return to the previous exam.
6. Change its assignment and try another possibility.
7. Continue until a complete valid timetable is found or all possibilities are exhausted.

Example: Suppose Mathematics and Computer Networks share students. If Mathematics is scheduled at 9 AM, Computer Networks cannot be scheduled at 9 AM. The algorithm tries another slot for Computer Networks. If no valid slot is available, it backtracks and changes an earlier assignment.

Backtracking is complete for a finite CSP when implemented correctly, but it can be slow without suitable heuristics and constraint propagation.

iii. Discuss how constraint propagation, such as forward checking, improves efficiency.

Forward checking reduces the domains of unassigned variables after each assignment.

Suppose Computer Networks has three possible slots:

[D(CN)={9AM,11AM,2PM}]

If Mathematics is assigned at 9 AM and both exams share students, forward checking removes 9 AM from the Computer Networks domain.

The updated domain becomes:

[D(CN)={11AM,2PM}]

If a variable's domain becomes empty, the algorithm detects the conflict immediately and backtracks instead of exploring further invalid assignments.

Other useful techniques include:

- Minimum Remaining Values (MRV): Choose the exam with the fewest available assignments first.
- Degree heuristic: Prefer exams that constrain the largest number of other exams.
- Least Constraining Value (LCV): Choose the slot that eliminates the fewest options for other exams.
- Arc consistency: Remove values that cannot participate in a consistent assignment with neighboring exams.

These techniques reduce unnecessary search and improve efficiency.

iv. Analyze real-world challenges and justify the best strategy for scalability.

Real-world challenges:

- Hundreds of courses and thousands of students.
- Limited examination halls and seating capacities.
- Changes in faculty availability.
- Different student accommodation requirements.
- Interacting constraints.
- Last-minute changes to courses, rooms, and dates.

Recommendation: CSP with backtracking, MRV, forward checking, and constraint propagation.

This combination is appropriate because it reduces the search space, detects conflicts early, and respects hard scheduling constraints.

For very large timetables, the university can use constraint programming or integer programming solvers, divide the problem into manageable groups where appropriate, and re-optimize affected exams when changes occur.

Conclusion: CSP is well suited to exam timetabling because it represents variables, domains, and constraints explicitly. Heuristics and constraint propagation make the process more scalable than basic backtracking alone.

5. Strategic Game AI for Real-Time Decision Making: Minimax and Alpha-Beta Pruning

i. Explain how the Minimax algorithm models this problem.

Minimax is a decision-making algorithm used in competitive games where one player tries to maximize the outcome and the opponent tries to minimize it.

In a strategy game:

- MAX player: The AI tries to choose the move with the highest utility.
- MIN player: The human opponent is assumed to choose moves that reduce the AI's utility.

The algorithm constructs a game tree representing possible moves and resulting game states.

Working:

1. Represent the current game state as the root of the tree.
2. Generate the possible moves.
3. Expand the tree to a selected depth.
4. Evaluate terminal states or leaf states using a utility function.
5. At MAX nodes, choose the highest child value.
6. At MIN nodes, choose the lowest child value.
7. Select the root move with the highest minimax value.

The recurrence is:

[V(s)= begin{cases} maxlimits_{a} V(s_a), & text{MAX} minlimits_{a} V(s_a), & text{MIN} end{cases}]

Here, (s) is a game state and (s_a) is the state reached after action (a).

Minimax assumes the opponent also makes rational decisions. This helps the AI select moves that remain strong against an intelligent opponent.

ii. Analyze the computational challenges of large game trees.

The main challenge is the large number of possible future states.

If the average branching factor is (b) and the search depth is (d), ordinary Minimax has approximately:

[O(b^d)]

time complexity.

For example, if each state has 10 possible moves and the AI searches 5 levels deep:

[10^5=100,000]

possible leaf nodes may need to be evaluated.

As search depth increases, the number of states grows exponentially.

Other challenges include:

- Limited time to choose a move.
- High memory requirements for storing the tree.
- Difficulty evaluating complex game states accurately.
- The need to respond to changing game conditions.
- The risk of missing important tactical moves when the search depth is too small.

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning.

Alpha-Beta pruning improves Minimax by avoiding branches that cannot influence the final decision.

It maintains two values:

- (alpha): The best value MAX can guarantee so far.
- (beta): The best value MIN can guarantee so far.

A branch can be pruned when:

[alphageqbeta]

Example: Consider a MAX node with two possible moves, A and B.

MAX

Move A

MIN

5

6

Value = min(5, 6) = 5

Move B

MIN

4

Pruned

Move B cannot beat A

Step-by-step reasoning:

1. The algorithm evaluates Move A and finds its value to be 5.
2. MAX now has an option worth 5, so (alpha=5).
3. It begins evaluating Move B, which is a MIN node.
4. The first child of B has a value of 4, so MIN can choose an outcome no greater than 4.
5. Therefore, (beta=4).
6. Since (alphageqbeta), or (5geq4), the remaining children of B can be pruned.
7. MAX selects Move A because Move B cannot produce a better outcome.

The key point is that pruning does not change the minimax decision when implemented correctly. It avoids unnecessary calculations.

Complexity:

- Minimax: (O(b^d)).
- Alpha-Beta, worst case: (O(b^d)).
- Alpha-Beta, with ideal move ordering: approximately (O(b^{d2})).

Good move ordering is important because evaluating promising moves first makes pruning more effective.

iv. Design an evaluation function and justify the factors considered.

An evaluation function estimates how favorable a game state is for the AI.

One possible function is:

[begin{aligned} E(s)={}&w_1M+w_2R+w_3T &+w_4P+w_5O-w_6D end{aligned}]

Where:

- (M) = military strength.
- (R) = available resources.
- (T) = territory controlled.
- (P) = progress toward the game's objectives.
- (O) = positional advantage.
- (D) = damage or strategic disadvantage.
- (w_1,ldots,w_6) = weights representing the importance of each factor.

Example: If the AI has more resources, controls important territory, and is close to achieving its objective, the evaluation score should increase. If it loses important units or territory, the score should decrease.

The weights depend on the game. A resource-focused game may assign a high weight to resources, while a combat-focused game may prioritize military strength.

The function should also consider immediate threats and imminent victories or defeats. Terminal states should receive sufficiently large positive or negative values so that winning is preferred over merely collecting resources.

v. Recommend a strategy for balancing optimality and real-time performance.

Recommendation: Iterative Deepening Minimax with Alpha-Beta Pruning and a time limit.

Working:

1. Start searching at depth 1.
2. Complete the search at that depth and record the best move.
3. Increase the depth gradually.
4. Use Alpha-Beta pruning to eliminate unnecessary branches.
5. Search promising moves first using move ordering.
6. Stop when the available time is nearly exhausted and return the best move from the last completed search.

Additional improvements include:

- Transposition tables: Store previously evaluated states to avoid repeated work.
- Quiescence search: Extend searches in unstable positions, such as active combat, to reduce misleading evaluations.
- Evaluation functions: Estimate the quality of states that cannot be searched to the end.
- Time management: Allocate more computation to critical decisions.

This approach does not guarantee a globally optimal move when the search is depth-limited, but it provides strong decisions within real-time limits.

Conclusion: Alpha-Beta pruning combined with iterative deepening offers a practical balance between decision quality and computational efficiency. It allows the AI to search deeper than ordinary Minimax within the same time budget.

Final comparison of all five scenarios

Scenario	**Main AI technique**	**Main advantage**

Emergency medicine drone	A* with real-time replanning	Balances route cost and estimated remaining cost
Smart city traffic	Genetic Algorithm with local refinement	Explores multiple timing plans
Mars rover	Online search with heuristic planning	Learns and adapts to unknown terrain
Exam timetable	CSP with backtracking and forward checking	Satisfies multiple constraints systematically
Strategy game	Iterative Deepening Minimax with Alpha-Beta	Improves decision quality within time limits

The main idea is to choose the AI algorithm according to the problem: pathfinding, optimization, exploration, constraint satisfaction, or adversarial decision-making. Combining algorithms is often more effective than relying on a single technique.